

Настройка и работа с TypeScript

TypeScript — это надстройка над языком JavaScript, которая компилируется в обычный JS-код, выполняемый в браузере, в Node.js или в другом окружении JavaScript, которое поддерживает стандарт ECMAScript 6.

Получение файлов конфигураций

Перед тем, как начать писать на TypeScript, нужно его сконфигурировать. Для этого выполните следующие действия:

1. (опционально) [Создайте](#) файл **package.json** в вашем репозитории:

```
npm init
```

Все команды нужно выполнять в корне репозитория, а не в корне вашего интерфейсного модуля. Файл **package.json** НЕ должен находиться на одном уровне с s3mod-файлами.

2. Установите утилиту **wasaby-cli**. Перед выполнением команды ниже замените строку [xx.yyyy] на версию нужной вам ветки разработки, например, 20.4000.

```
npm install git+ssh://git@git.sbis.ru/saby/wasaby-cli.git#rc-[xx.yyyy] --save-dev
```

3. В файле **package.json**, добавьте следующие строки в раздел **scripts**:

```
1  "scripts": {  
2    ...  
3    "start": "wasaby-cli initTSEnv",  
4    "update-cli-store": "wasaby-cli loadProject",  
5    "build": "wasaby-cli buildProject"  
6  }
```

В данном примере:

- **start** настраивает окружение, выгружает зависимости и запускает разводящую с демо-примерами (если они у вас есть);
- **update-cli-store** обновляет зависимости, указанные в ваших s3mod-файлах;
- **build** компилирует ваши исходные файлы (ts в js, less в css и т.п.). Эту команду можно выполнять только если предварительно было настроено окружение и выгружены зависимости (например, с помощью команды **start**);

4. Запустите подготовку окружения через скрипт **start**:

```
npm run start
```

Скрипт установит рекомендуемые [конфигурации компилятора](#) TypeScript и линтера ([TSLint](#) или [ESLint](#) в зависимости от репозитория), добавив в корень проекта необходимые файлы.

5. В файл с именем **.gitignore** автоматически добавляются все файлы и каталоги, которые появились в проекте в результате подготовки окружения (они не нужны в вашем git-репозитории).

Работа с локальным стендом

Вы можете указать параметры **baseUrl** и **outDir** в файле **tsconfig.json** и при работе с [локальным стендом](#). Это даст вам следующие возможности:

- IDE будет "видеть" все интерфейсные модули приложения, которое у вас развернуто. Будут работать подсказки и переходы.
- Вы сможете выполнять компиляцию TS средствами IDE (актуально только для [режима отладки](#)).

Для этого укажите каталог ресурсов внутри каталога, в который развернут локальный стенд. Например:

```
1 "compilerOptions": {
2   "baseUrl": "c:/stands/online-201000/build-ui/resources",
3   "outDir": "c:/stands/online-201000/build-ui/resources",
4   //...
5 }
```

Tips and Tricks

Зависимости TS-модуля и результат компиляции

Зачастую разработчики в своих ts-файлах импортируют внешние зависимости.

Иногда зависимости не попадают в результат компиляции, это происходит в двух случаях:

- Когда подключенная зависимость никак не используется (при компиляции файла импорт этой зависимости пропадёт);
- Когда подключенная зависимость используется только в декларациях типов (все декларации таких типов будут удалены при компиляции, а зависимости, используемые только для объявления типов, также не попадут в результат компиляции).

Пример (зависимость будет удалена):

```
1 import {Record} from 'Types/type';
2 export function handleRecord(rec: Record) {...}
```

В данном примере **Record** используется только для объявления типа аргумента, поэтому компилятор удалит такую зависимость.

Пример (зависимость попадет в результат компиляции):

```
1 import {Record} from 'Types/type';
2 export function createRecord() {
3   return new Record();
4 }
```

В этом примере **Record** используется для создания экземпляра во время выполнения кода приложения, поэтому компилятор оставит зависимость от модуля, из которого был получен Record.

Импорт AMD-модуля в TS-модуль

Сейчас Платформа находится в процессе перевода на TypeScript, в связи с этим часть модулей еще не переписана и, если существует необходимость притянуть из платформы еще не переписанный модуль, используйте специальный синтаксис **import** для подключения AMD-модулей:

```
import MyModule = require('NameOf/Some/Amd/Module');
```

Динамические зависимости в TS-модуле

Иногда в силу ряда причин разработчик не может подключить сразу все зависимости модуля статически. Есть возможность подключать некоторые зависимости динамически, например при наступлении какого-либо пользовательского сценария.

Для этого следует использовать [dynamic imports](#), которые описаны в стандарте ES6:

```
1 import('Types/collection').then((collection) => {
2   let list = new collection.List();
3   //Do something with list
4 }).catch((err) => {
5   //Do something with err
6 });
```

Зависимости TS-модуля для side effect

Существуют задачи импортировать что-то для создания side effect'a, например для загрузки стилей через css-плагин. Для решения таких задач используйте синтаксис **side effects**:

```
import 'css!Some/Module' ;
```

См. также

- [Юнит-тестирование](#)
- [Стандарт разработки на TypeScript](#)